Handling signals

- A process can specify what to do when a particular signal is received.
- In particular, the `signal()` system call may be used to override the default kernel-supplied action (or handler).

`signal()`

Synopsis: `void(*signal(int signo,void(*func)(int)))(int)`

This system call has two arguments:

- `int signo`: the signal number to be reprogrammed, a constant from the previous table.
- `void (*func)(int)`: the address of a user-defined function, or `SIG_IGN` to ignore a signal (except `SIGKILL` and `SIGSTOP`), or `SIG_DFL` to use the default handler.

About the prototype of `signal()`

`void (*signal(...))`$_1$`(int)`$_2$`;`

- The pair $(\ )_1$ is there because `signal()` returns a function, not a value.
- $(int)_2$ is there because `signal()` returns a function that takes one parameter of type `int`.
- Because of the `*`, `signal()` returns the address(pointer) of a function.

What does the `signal()` system call exactly return?

- If successful, `signal()` returns the previous `func()` value associated with `signo` (the previous handler) or,
- `SIG_ERR` (i.e. -1) otherwise.

Example: ignoring CTRL-C and CTR-Z

```
int main(int argc, char *argv[]){
  int i;
  void (*oldHandler1)(); //to save default handlers
  void (*oldHandler2)(); //for CTRL-C and CTRL-Z

  oldHandler1=signal(2, SIG_IGN);  //ignore CTRL-C
  oldHandler2=signal(24, SIG_IGN); // ignore CTRL-Z
  for(i=1; i<=10; i++){
    printf("I am not sensitive to CTRL-C/CTRL-Z\n");
    sleep(1);
  }
  signal(2, oldHandler1);  // restore default
  signal(24, oldHandler2); // restore default
  for(i=1; i<=10; i++){
    printf("I am sensitive to CTRL-C/CTRL-Z\n");
    sleep(1);
  }
}
```

Example 1: replacing a default handler by our own one-time handler

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void alarm_handler(int);

void alarm_handler(int dummy){
  printf("I got an alarm, I took care of signal %d\n", dummy);
  alarm(3);
}

int main(int argc, char *argv[]){

  signal(14, alarm_handler);//install handler
  alarm(3);

  while(1){
    printf("I am working\n");
    sleep(1);
  }
}
```

Example 1: replacing a default handler by our own one-time handler

Output of the previous example

```
I am working
I am working
I am working
I got an alarm, I took care of signal 14
I am working
I am working
I am working
Alarm clock
```

Example 2: replacing a default handler by our own

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void alarm_handler(int);

void alarm_handler(int dummy){
  printf("I got an alarm, I took care of signal %d\n", dummy);
  signal(14, alarm_handler); //install handler
  alarm(3);
}

int main(int argc, char *argv[]){

  signal(14, alarm_handler);//install handler
  alarm(3);                 // for first time

  while(1){
    printf("I am working\n");
    sleep(1);
  }
}
```

Example 2: replacing a default handler by our own

Output of the previous example

```
I am working
I am working
I am working
I got an alarm, I took care of signal 14
I am working
I am working
I am working
I got an alarm, I took care of signal 14
I am working
I am working
I am working
I got an alarm, I took care of signal 14
I am working
I am working
...
```

signal() and fork()

A child process inherits the signal setting from its parent during a `fork()`.

```
int main(int argc, char *argv[]){
  int i;
  void (*oldHandler1)(); //to save default handlers
  void (*oldHandler2)(); //for CTRL-C and CTRL-Z

  oldHandler1=signal(2, SIG_IGN);  //ignore CTRL-C
  oldHandler2=signal(24, SIG_IGN); // ignore CTRL-Z

  if(fork()!=0)
    for(i=1; i<=10; i++){
      printf("Parent: I am not sensitive to CTRL-C/CTRL-Z\n");
      sleep(1);
  }
 else while(1){
        printf("Child: I am not sensitive to CTRL-C/CTRL-Z?\n");
        sleep(1);
      }
}
```

Both the child and parent are insensitive to `CTRL-C/CTRL-Z`.

`signal()` and `exec()`

When a process performs an `exec()`,

- previously ignored signals remain ignored,
- installed handlers are set back to the default handler.

Stacked vs. non stacked signals

Except `SIGCHLD`, signals are not stacked. Therefore, if a number of signals are sent to a process only one of the signals is actually processed.

Suspending a process

`pause()`

Synopsis: `int pause(void);`

- The `pause()` suspends the calling process until it receives a signal.
- A call to `pause()` does not return anything useful.
- The signal must be one that is not currently set to be ignored by the calling process.
- `pause()` is typically used to wait for an alarm signal.

Example: pause()

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void AlarmHandler(int);

void AlarmHandler(int dummy){
  static int n = 0;
  if(n++ < 6 ){
    printf("Beeping%c%c%c\n", 7, 7, 7);
    signal(14, AlarmHandler); //install a handler
    alarm(10);
  }else
    exit(0);
}

int main(int argc, char *argv[]){
  alarm(10);
  signal(14, AlarmHandler);

  while(1)
     pause();
}
```

Sending a signal to a process

`kill()`

Synopsis: `int kill(pid_t pid, int signo);`

- sends the signal `signo` to a process or a group of processes, defined by `pid`.
- returns 0 on success, -1 otherwise.

The signal is sent only when at least one of the following conditions is satisfied:

- The sending and receiving processes have the same owner.
- The sending process is owned by a super-user.

Example

```
kill(2344, SIGTERM);
```

To terminate process 2344.

The `pid` parameter in `kill()` can take several values with different meanings:

- If `pid > 1`, the signal is sent to the process with id `pid`.
- If `pid` is 0, the signal is sent to all processes in the sender's process group.
- If `pid` is -1 then
  - if the sender is owned by a super-user, the signal is sent to all processes, including the sender.
  - otherwise, the signal is sent to all processes owned by the same owner as the sender.
- If `pid` is equal to `-n`, with `n > 1`, the signal is sent to all processes with a process group-id equal to `n`.

Example: Parent-Child synchronization I

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void myAlarmHandler(int);

void myAlarmHandler(int dummy){}; // to avoid quitting

int main(int argc, char *argv[]){
  pid_t pid;

  if((pid=fork()) > 0){
    printf("My child should wait until I am done\n");
    sleep(4);
    printf("Child, now you can do your job\n");
    kill(pid, SIGALRM);  // let the child wake up
    printf("Parent Exiting\n");
  }
  else{
    printf("I have to wait for my parent\n");
    signal(SIGALRM,  myAlarmHandler);
    pause();
    printf("OK, now I can do my job\n");
    sleep(2);
    printf("Child Exiting\n");
  }
  exit(0);
}
```

Example: Parent-Child synchronization I

Output of the previous program

```
My child should wait until I am done
I have to wait for my parent
Child, now you can do your job
Parent Exiting
OK, now I can do my job
Child Exiting
```

Example: Parent-Child synchronization II

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void action(int);

void action(int dummy){
  sleep(1);
  printf("Switching\n");
}

int main(int argc, char *argv[]){
  pid_t pid;

  if((pid=fork())>0){//parent
    sleep(1);
    while(1){
      printf("Parent is running\n");
      kill(pid, SIGUSR1);
      signal(SIGUSR1, action);
      pause();
    }
  }
  else
      while(1){//child
        signal(SIGUSR1, action);
        pause();
        printf("Child is running\n");
        kill(getppid(), SIGUSR1);
      }
}
```

Example: Parent-Child synchronization II

Output of the previous program

```
Parent is running
Switching
Child is running
Switching
Parent is running
Switching
Child is running
Switching
Parent is running
Switching
Child is running
...
```

Example: a non-blocking `wait()`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void childDeath(int dummy){
  pid_t pid;
  int status;

  pid = wait(&status);
  printf("Child %d has terminated\n", pid);
}

int main(int argc, char *argv[]){
  pid_t pid;

  while(1){
    signal(SIGCHLD, childDeath);
    if((pid=fork())==0){
      sleep(random()%20);
      exit(0);
    }
    printf("Created a child, pid=%d\n", pid);
    sleep(2);
  }
}
```

Process groups: review

Process groups: review

Unix organizes processes as fellows:

- Every process is a member of a `process group`.
- A child inherits its `process group` from its parent.
- When a process `exec`s, its `process group` remains the same.
- A process may change its `process group` to a new value using `setpgid()`. See lecture on Process Control for more details.

Control terminal

- Every process may have an associated `control terminal`.
- A child inherits its `control terminal` from its parent.
- A process `execs`, its `control terminal` remains the same.
- Every terminal is associated with a `control process`, a piece of software that manages the terminal.

The shell and control terminals

When an interactive shell starts, it is the control process of its control terminal:

Executing a foreground process

- When a shell executes a foreground process, the child shell changes its `group process` number to its `pid`, `execs` the command and, takes control of the terminal.
- In this case, signals generated from the terminal go to the command and not to the parent shell.
- When the command terminates, the parent shell takes back the control of the terminal.

Executing background process

- When a shell executes a background process, the child shell changes its `group process` number to its `pid` then `execs` the command. However, it does not take control of the terminal.
- In this case, signals generated from the terminal go to the original parent shell which is still the terminal's control process.

Example 1: CTRL-C goes to all processes in a process group

When CTRL-C is typed, SIGINT is sent to all processes in the process group of the terminal's control process.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void CTRL_handler(int);

void CTRL_handler(int dummy){
  printf("Process %d got a CTRL-C, exit\n", getpid());
  exit(2);
}

int main(int argc, char *argv[]){
  int i;
  printf("First process, PID=%d, PPID=%d, PGID=%d\n",
         getpid(), getppid(), getpgid(0));
  signal(SIGINT, CTRL_handler);
  for(i=1; i<=3; i++)
    fork();
  printf("PID=%d PGID=%d\n", getpid(), getpgid(0));
  pause();
}
```

Output of example 1 running foreground

CTRL-C is pressed after all the processes are created:

```
First process, PID=3928, PPID=3885, PGID=3928
PID=3928 PGID=3928
PID=3931 PGID=3928
PID=3929 PGID=3928
PID=3930 PGID=3928
PID=3932 PGID=3928
PID=3933 PGID=3928
PID=3934 PGID=3928
PID=3935 PGID=3928
^CProcess 3932 got a CTRL-C, exit
Process 3930 got a CTRL-C, exit
Process 3929 got a CTRL-C, exit
Process 3935 got a CTRL-C, exit
Process 3931 got a CTRL-C, exit
Process 3928 got a CTRL-C, exit
Process 3934 got a CTRL-C, exit
Process 3933 got a CTRL-C, exit
```

Example 2: CTRL-C

```
void CTRL_handler(int dummy){
  printf("Process%d got CTR-C, exiting\n", getpid());
  exit(0);
}

int main(int argc, char *argv[]){
  int i, pid;

  signal(2, CTRL_handler);
  if((pid=fork()) == 0){
   setpgid(0, getpid());//child is in its own group
   printf("PID=%d, PGID=%d\n",getpid(),getpgid(0));
  }else
    printf("PID=%d, PGID=%d\n", getpid(), getpgid(0));

  for(i=1; i<=10; i++){
    printf("Process %d is still alive\n", getpid());
    sleep(2);
  }
}
```

What happens when we press CTRL-C?



Handling Signal

- A signal is a message from one process to another.
- Signal are sometime called “software interrupt”
- Signals usually occur asynchronously.
- Signals can be sent

  A. by one process to anther (or to itself)

  B. by the kernel to a process.
- Unix signals are content-free. That is the only thing that can be said about a signal is “it has arrived or not”

Handling Signal

- Most signals have predefined meanings:
  A. sighup (HangUp): when a terminal is closed, the hangup signal is sent to every process in control terminal.
  B. sigint (interrupt): ask politely a process to terminate.
  C. sigquit (quit): ask a process to terminate and produce a codedump.
  D. sigkill (kill): force a process to terminate.
- See signEx1.c

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
main() {
   int fpid, *status;
   printf("Before forking ...\n");
   fpid = fork();
   if (fpid == 0) {
      printf("Child Process fpid=%d pid=%d ppid=%d\n",
                                          fpid, getpid(), getppid());
      for(;;); /* loop forever */
   } else {
      printf("Parent Process fpid=%d pid=%d ppid=%d\n",
                                          fpid, getpid(), getppid());
   }
   wait(status); /* wait for child process */
   printf("After forking fpid=%d pid=%d ppid=%d\n",
                                          fpid, getpid(), getppid());
}
```

```
$ cc sigEx1.c -o sigEx1
$ sigEx1 &
Before forking ...
Parent Process fpid=14989 pid=14988 ppid=14759
Child Process fpid=0 pid=14989 ppid=14988
$ ps
   PID TTY      TIME CMD
 14988 pts/9    0:00 sigEx1
 14759 pts/9    0:01 tcsh
 14989 pts/9    0:09 sigEx1
$ kill -9 14989
$ ps
   ...
```

Programs, daemons and libraries

- As we earlier stated, processes are programs executing in the system. Processes are, however, also a bit more than that: They also include set of resources such as open files, pending signals, internal kernel data, processor state, an address space, one or more threads of execution, and a data section containing global variables. Processes, in effect, are the result of running program code. A program itself is not a process. A process is an *active* program and related resources.
- Threads are objects of activity inside the process. Each thread contains a program counter, process stack and processor registers unique to that thread. Threads require less overhead than forking a new process because the system does not initialize a new system virtual memory space and environment for the process. Threads are most effective on multi-processor or multi-core systems where the process flow can be scheduled to run on another processor thus gaining speed through parallel or distributed processing.
- Daemons are processes that run in the background unobtrusively, without any direct control from a user (by disassociating the daemon process from the controlling TTY). Daemons can perform various scheduled and non-scheduled tasks and often they serve the function of responding to the requests from other computers over a network, other programs or hardware activity. Daemons have usually init as their parent process, as daemons are launched by forking a child and letting the parent process die, in which case init adopts the process. Some examples of the daemons are: httpd (web server), lpd (printing service), cron (command scheduler).

Programs, daemons and libraries

- *Linking* refers to combining a program and its libraries into a single executable and resolving the symbolic names of variables and functions into their resulting addresses. Libraries are a collection of commonly used functions combined into a package.
- There are few types of linking:
- *Static linking*: Static linking copies a set of routines to the executable target application, thus expanding the target executable size.
- *Dynamic linking*: Dynamic linking means that the subroutines of a library are loaded into an application program at runtime. Dynamic libraries remain on file-system as separate files from executable. Most of the work of linking is done at the time the application is loaded, thus creating a bit of overhead in application startup time.
- *Run-time linking*: Run-time linking (or dynamic loading) is a subset of dynamic linking where a dynamically linked library loads or unloads at run-time on request.
- In addition to being loaded statically or dynamically, libraries can also be shared. Dynamic libraries are almost always shared, static libraries can not be shared at all. Sharing allows same library to be used by multiple programs at the same time, sharing the code in memory.

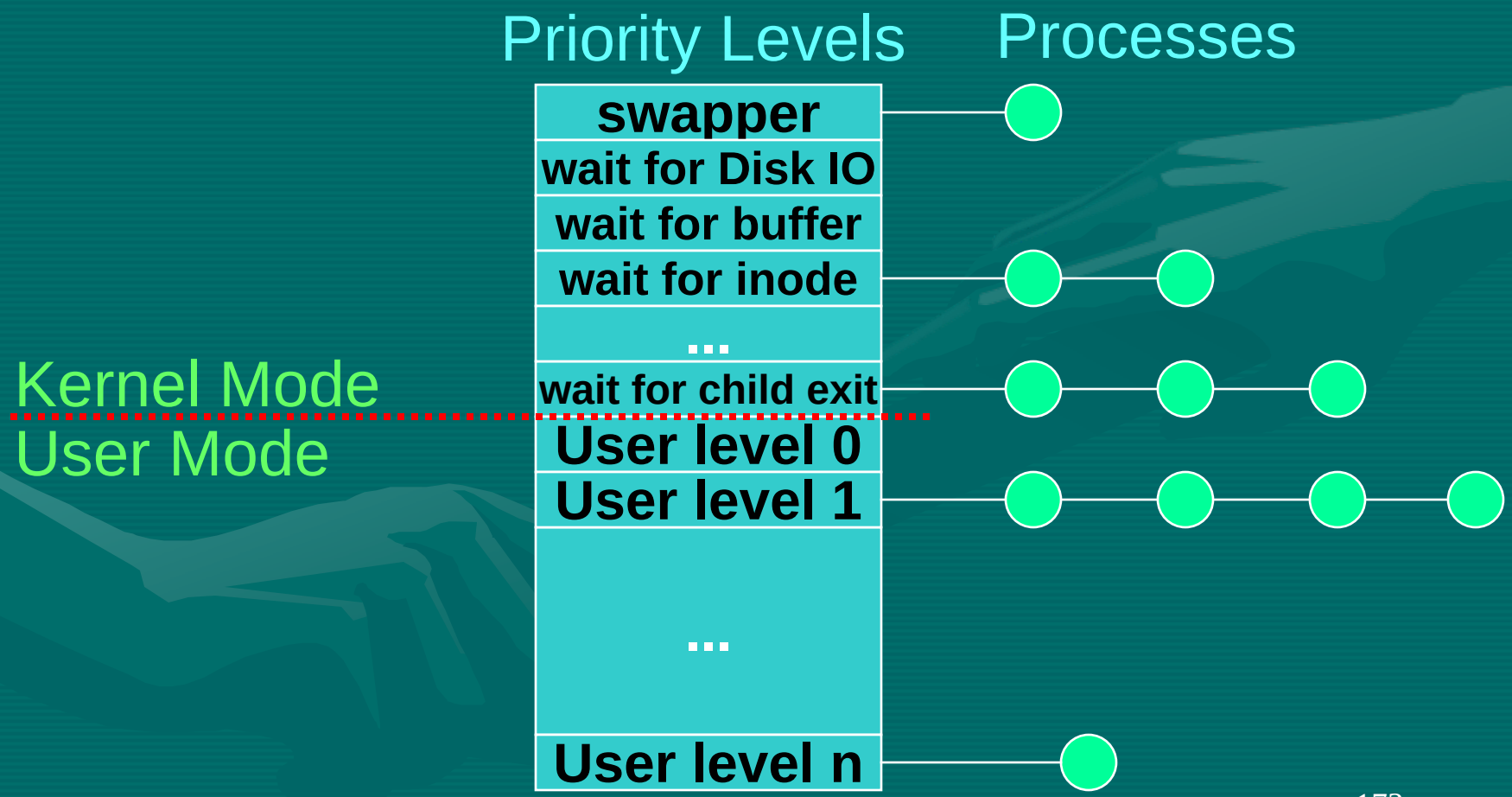
Programs, daemons and libraries

- Dynamic linking of shared libraries provides multiple benefits:
  - Common code of the applications is shared at runtime, which reduces the total memory usage.
  - Application executable file size is reduced, which reduced the total storage usage.
  - Fixing a bug from shared code fixes the bug from all the applications using the same code.
- Most Linux systems use almost entirely dynamic libraries and dynamic linking.

Scheduling Processes

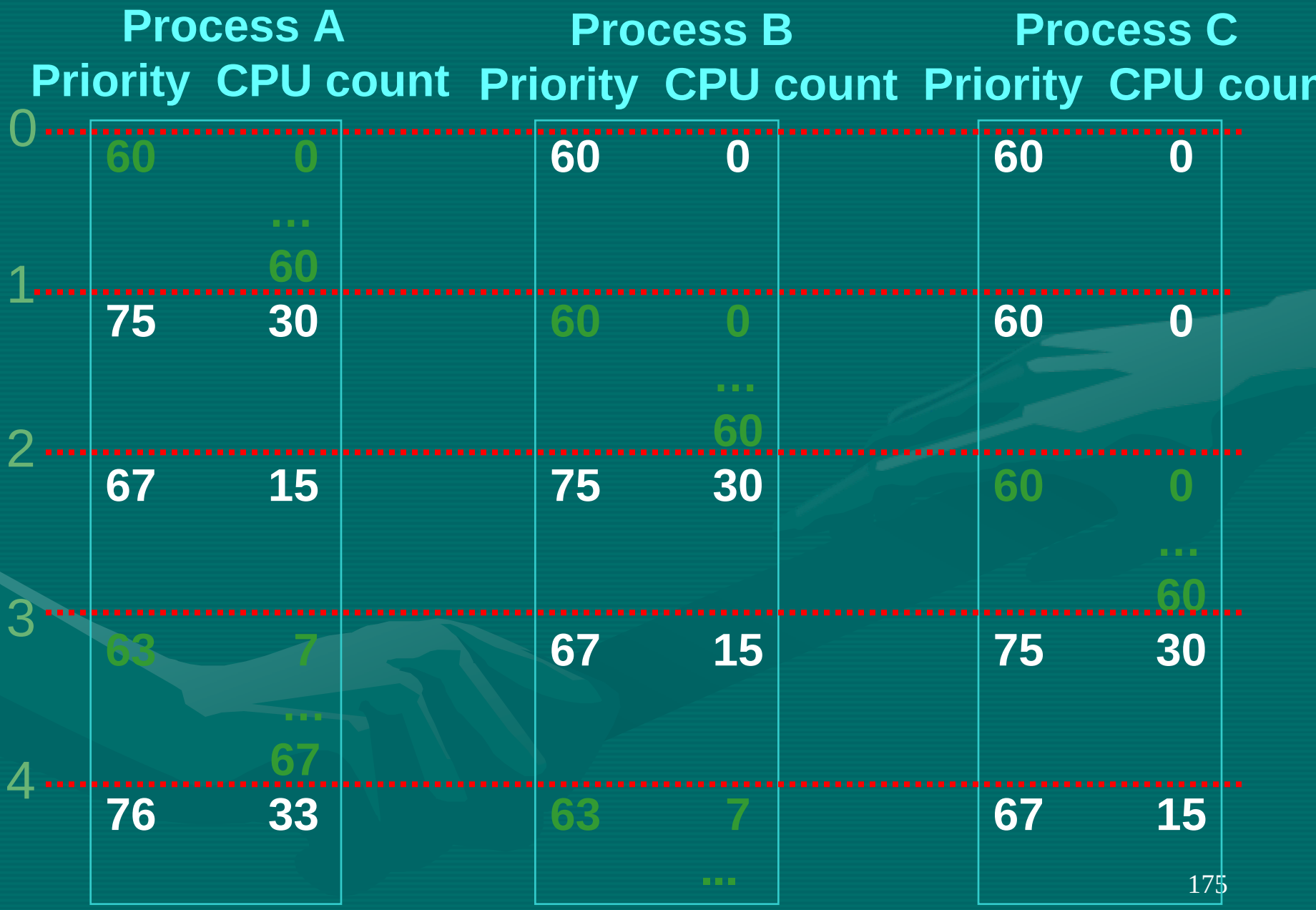
- On a time sharing system, the kernel allocates the CPU to a process for a period of time (time slice or time quantum) preempts the process and schedules another one when time slice expired, and reschedules the process to continue execution at a later time.
- The scheduler use round-robin with multilevel feedback algorithm to choose which process to be executed:
  A. Kernel allocates the CPU to a process for a time slice.
  B. preempts a process that exceeds its time slice.
  C. feeds it back into one of the several priority queues.

Process Priority



Process Scheduling (Unix System V)

- There are 3 processes A, B, C under the following assumptions:
  A. they are created simultaneously with initial priority 60.
  B. the clock interrupt the system 60 times per second.
  C. these processes make no system call.
  D. No other process are ready to run
  E. CPU usage calculation: CPU = decay(CPU) = CPU/2
  F. Process priority calculation: priority = CPU/2 + 60.
  G. Rescheduling Calculation is done once per second.


Process A
Priority CPU count
Process B
Priority CPU count
Process C
Priority CPU count
0
1
2
3
4
60 0
...
60
75 30
67 15
63 7
...
67
76 33
60 0
60 0
...
60
75 30
63 7
...
60 0
60 0
60 0
...
60
75 30
67 15

Linux System Programming

Part 2

Booting

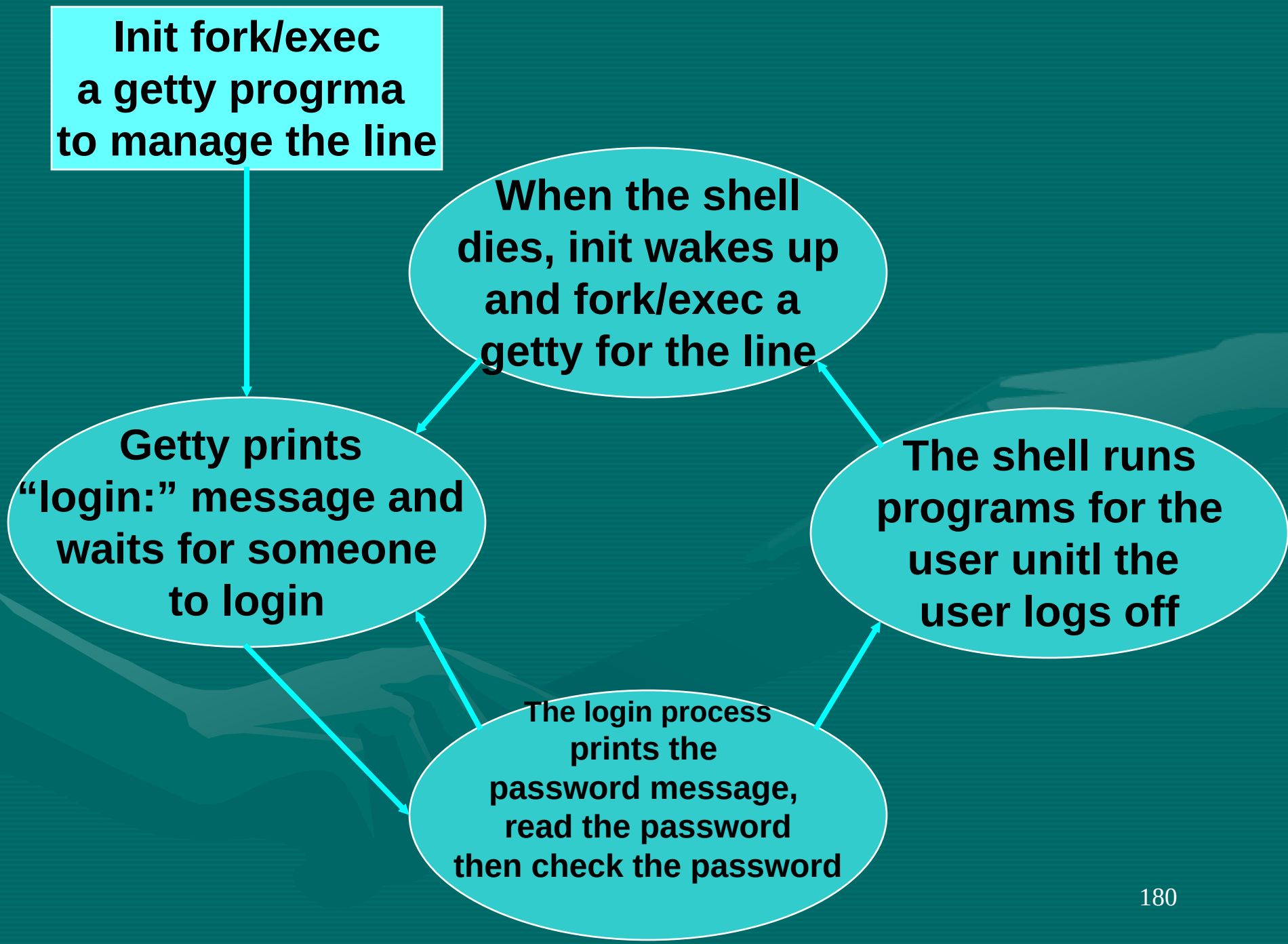
- When the computer is powered on or rebooted, a short built-in program (maybe store in ROM) reads the first block or two of the disk into memory. These blocks contain a loader program, which was placed on the disk when disk is formatted.
- The loader is started. The loader searches the root directory for /linux or /root/linux and load the file into memory
- The kernel starts to execute.

The first processes

- The kernel initializes its internal data structures:
  it constructs linked list of free inodes, regions, page table
- The kernel creates u area and initializes slot 0 of process table
- Process 0 is created
- Process 0 forks, invoking the fork algorithm directly from the Kernel. Process 1 is created.
- In kernel mode, Process 1 creates user-level context (regions) and copy code (/etc/init) to the new region.
- Process 1 calls exec (executes init).

init process

- The init process is a process dispatcher: spawning processes, allow users to login.
- Init reads /etc/inittab and spawns getty
- when a user login successfully, getty goes through a login procedure and execs a login shell.
- Init executes the wait system call, monitoring the death of its child processes and the death of orphaned processes by exiting parent.


Init fork/exec
a getty progrma
to manage the line
When the shell
dies, init wakes up
and fork/exec a
getty for the line
Getty prints
“login:” message and
waits for someone
to login
The shell runs
programs for the
user unitl the
user logs off
The login process
prints the
password message,
read the password
then check the password

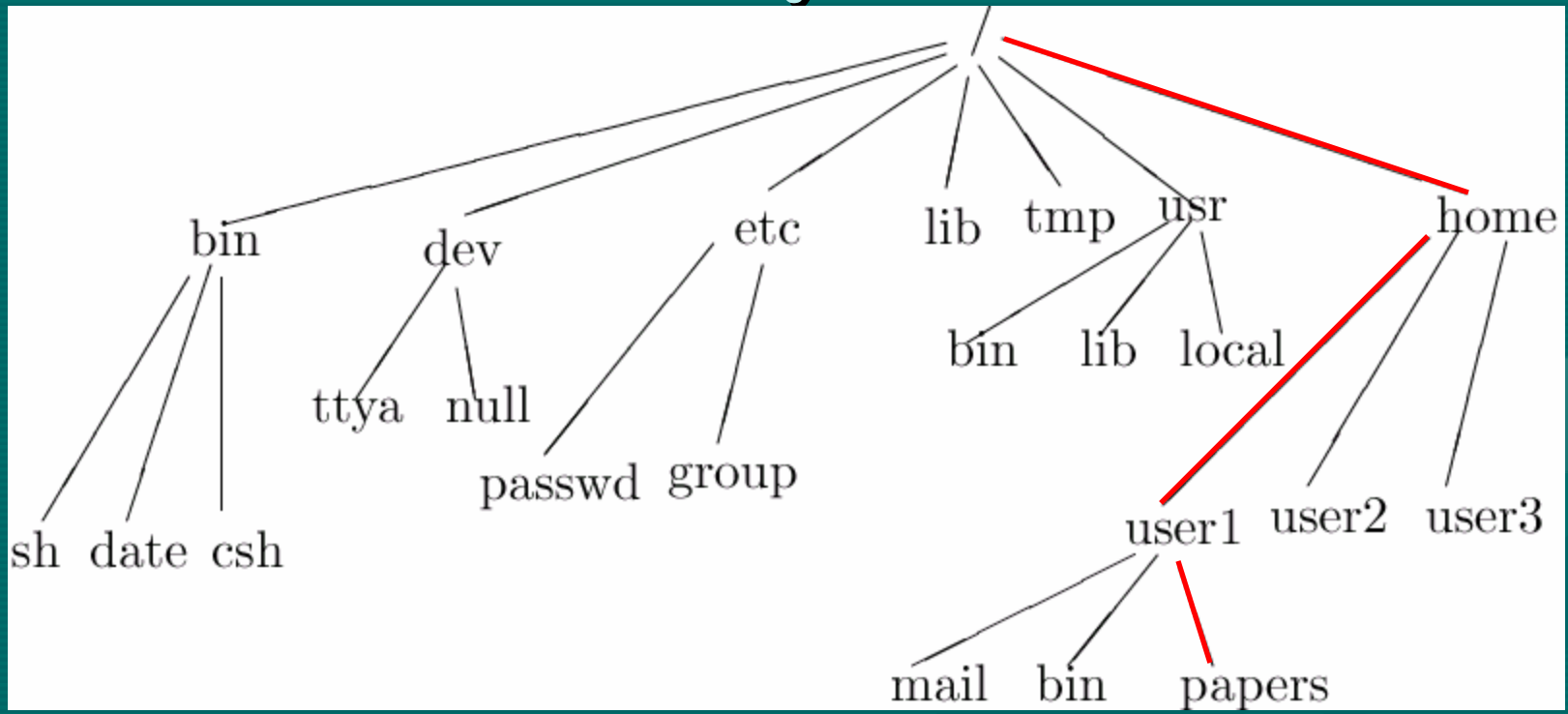
File Subsystem

- A file system is a collection of files and directories on a disk or tape in standard UNIX file system format.
- Each UNIX file system contains four major parts:
  A. boot block:
  B. superblock:
  C. i-node table:
  D. data block: file storage

File System

- Linux files are organized by a hierarchy of labels, commonly known as a directory structure. The files referenced by these labels may be of three kinds:
  - Regular files, which contains a sequence of bytes that generally corresponds to code (programs) or data.
  - Directory files, which are stored on disk in a special format and form the backbone of the file system
  - Special file, which correspond to peripherals such as printers or disks.
- To the user, Linux file system appears as a hierarchical directory tree obeying UNIX semantics.

File System



- / is the root directory; reference point for all directories. Every file has a unambiguous pathname:
  - /home/user1/papers

Some directories found in UNIX systems

- /bin Binaries which are absolutely essential to run Linux.
- /boot All the files required for booting Linux on a system.
- /dev All the devices have their corresponding files here.
- /etc All the configuration files for the various software are stored here. Don't play with this directory.
- /home All users will have their 'My Documents' under this directory. If your id is ravi, your 'My Documents' (called home-directory) is /home/ravi.
- /lib The libraries required by system-applications. (Just like DLLs in Windows.)
- /lost+found When a disk-check finds files which are damaged or which are not linked to any directory, they are recovered to this directory. Such damages are almost always due to incorrect shutdown.

Some directories found in UNIX systems

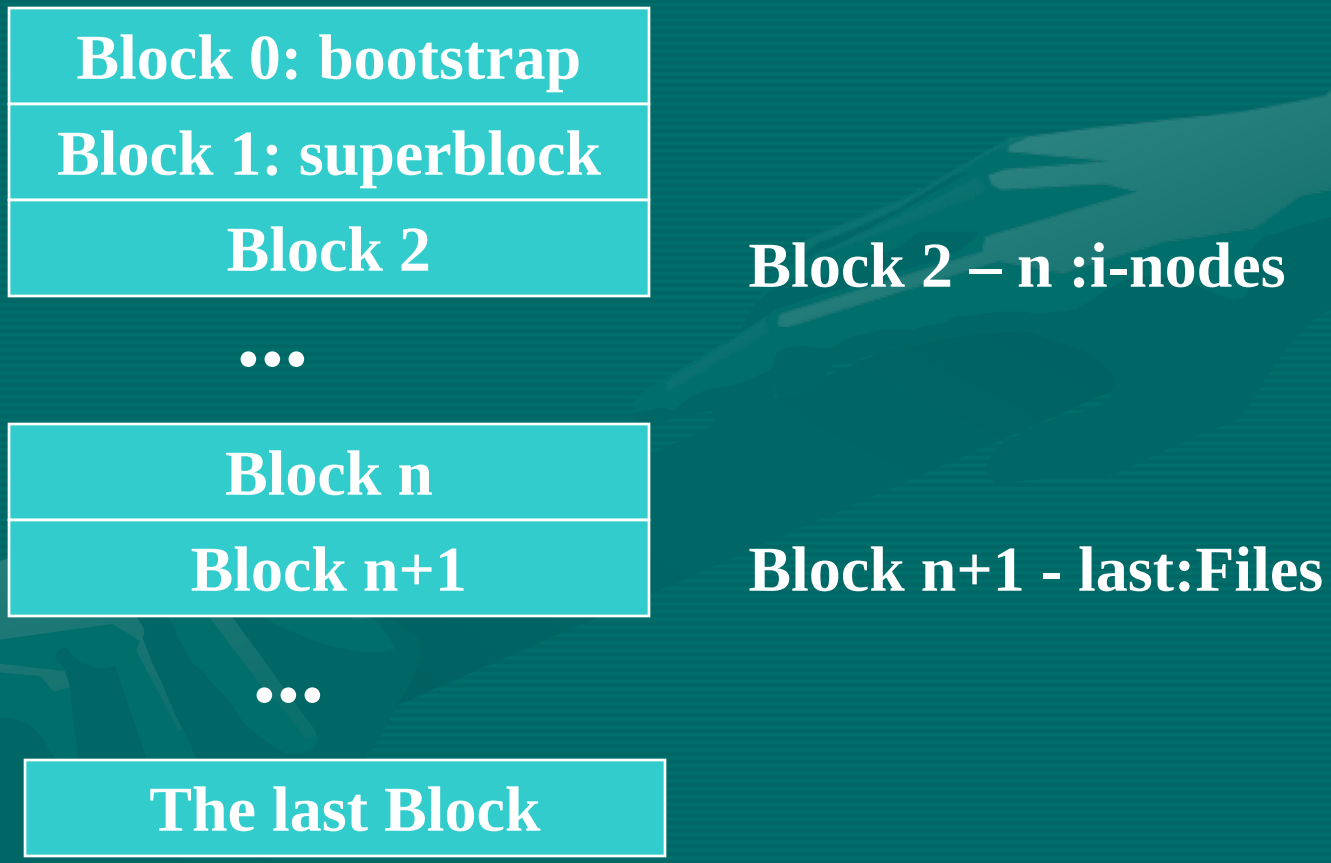
- /mnt The directory where peripherals and other file-systems are mounted.
- /opt The directory where optional software are installed.
- /proc proc houses a pseudo-file system. Its contents really do not exist anywhere on the disk, and are made available only when you cd to this directory and look at some file.
- /root The home-directory for the super-user: root.
- /sbin The system-administration binaries exist here.
- /tmp The directory where temporary files are created and stored. All users can save files here.
- /usr Everything related to users
- /var Files whose contents vary frequently are in this directory.

Some directories found in UNIX systems

- /usr Everything related to users
  - /usr/bin houses critical binaries of the users
  - /usr/include The header-files required by programs for compilation.
  - /usr/lib The libraries required by user-applications.
  - /usr/local Files peculiar to this particular machine.
  - /usr/sbin User-administration binaries.
  - /usr/share Information that can be shared by most users.
  - /usr/src The source-code for the Linux kernel.
  - /usr/X11R6 Files needed by the X Window system.

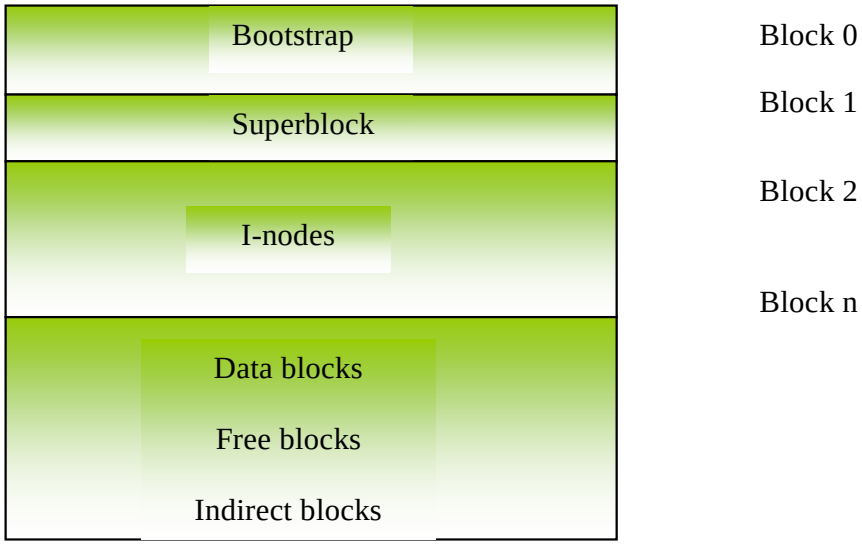
- /var Files whose contents vary frequently are in this directory.
  - /var/log The log-files of the system.
  - /var/spool Directories for mail, news, printing and other queued work.

File System Layout



Disk Layout

The Physical Layout of a Unix disk



Boot Block

- A boot block may contains several physical blocks.
- Note that a physical block contains 512 bytes (or 1K or 2KB)
- A boot block contains a short loader program for booting
- It is blank on other file systems.

Superblock

- Superblock contains key information about a file system
- Superblock information:

  A. Size of a file system and status:

  label: name of this file system

  size: the number of logic blocks

  date: the last modification date of super block.

  B. information of i-nodes

  the number of i-nodes

  the number of free i-nodes

  C. information of data block: free data blocks.
- The information of a superblock is loaded into memory.

I-nodes

- i-node: index node (information node)
- i-list: the list of i-nodes
- i-number: the index of i-list.
- The size of an i-node: 64 bytes.
- i-node 0 is reserved.
- i-node 1 is the root directory.
- i-node structure: next page

Files and inodes

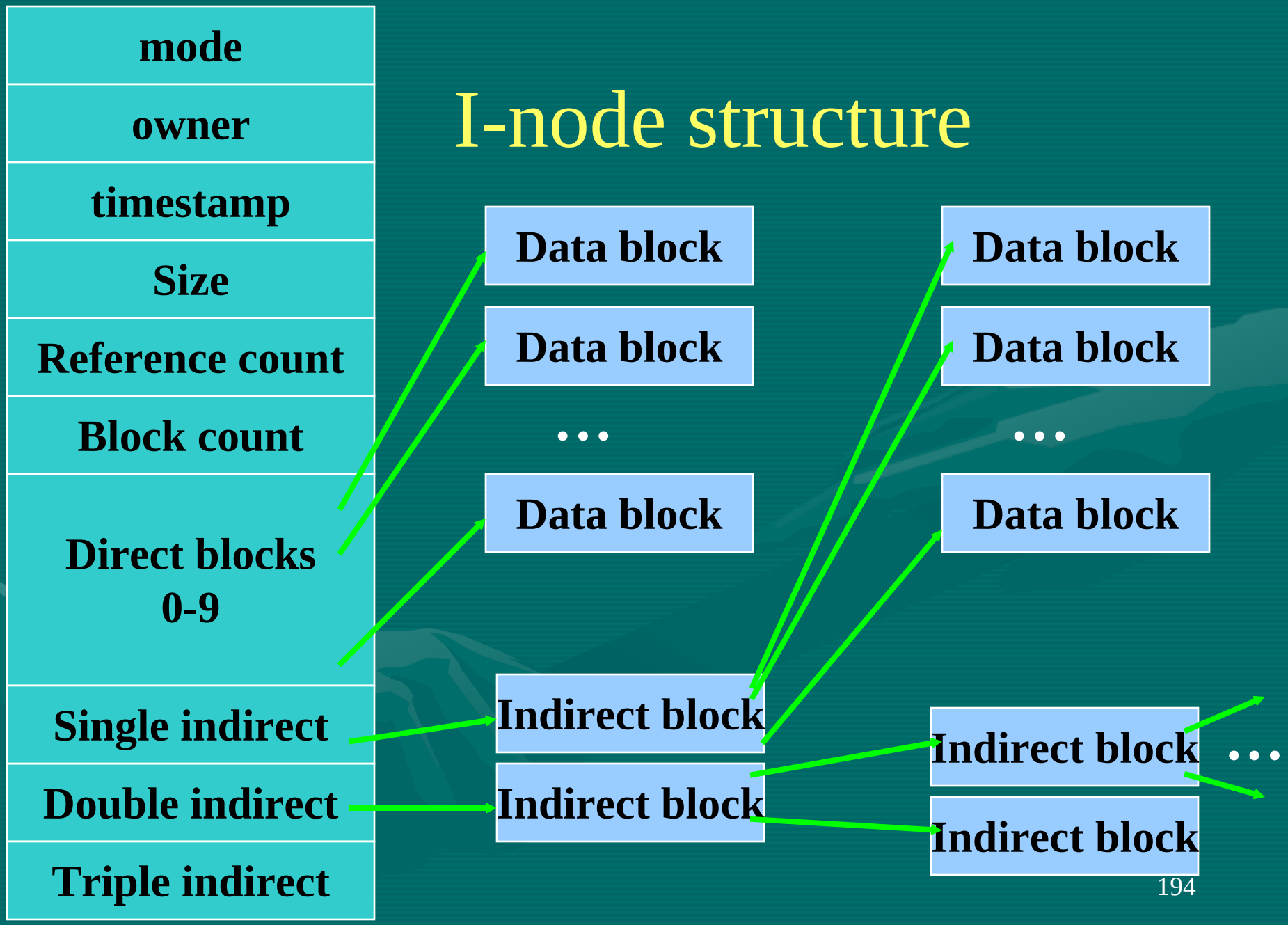
- When a file-system is created to partition, a data structures containing information about files are written into it. These structures are called *inodes*. Each file in file-system has an inode, identified by inode serial number.
- *Inode* contains the following information of the file:
  - *Owner:* Owner of the file
  - *Group:* Group owner of the file
  - *Permissions:* File permissions, more on these to follow
  - *Last* read time: Time when file was last accessed (atime)
  - *Last* change time: Time when file was last modified (mtime)
  - *Inode* change time: Time when inode itself was last modified (ctime)
  - *Hard* links: Number of hard links to this file
  - *Size:* The length of the file in bytes
  - *Address:* Pointer to the actual content data of the file
- The only information not included in an inode is the file name and directory. These are stored in the special directory files, each of which contains one filename and one inode number. The kernel can search a directory, looking for a particular filename, and by using the inode number the actual content of the inode can be found, thus allowing the content of the file to be found.
- Inode information can be queried (viewed) from the file using *stat* command:

```
$stat /etc/passwd
```

Hard and Soft Linking

- Notice that the stat command shown above is from desktop Linux.
- Some advantages of inodes:
- Multiple filenames can be associated with the same inode. This is called *hard linking* or just linking. Notice that hard links only work within one partition as the inode numbers are only unique within a given partition.
- As processes open files, the kernel converts the filename to an inode number at the first possible chance, thus moving the file (within same partition) does not prevent a process from accessing the file once it is opened (as long as the process knows the inode number).
- In addition to hard links, Linux file-system also supports soft links, or more commonly called, symbolic links (or for short: symlinks). A symbolic link contains the path to the target file instead of a physical location on the hard disk. Since symlinks do not use inodes, symlinks can span across the partitions. Symlinks are transparent to processes which read or write to files pointed by a symlink, process behaves as if operating directly on the target file.

I-node structure

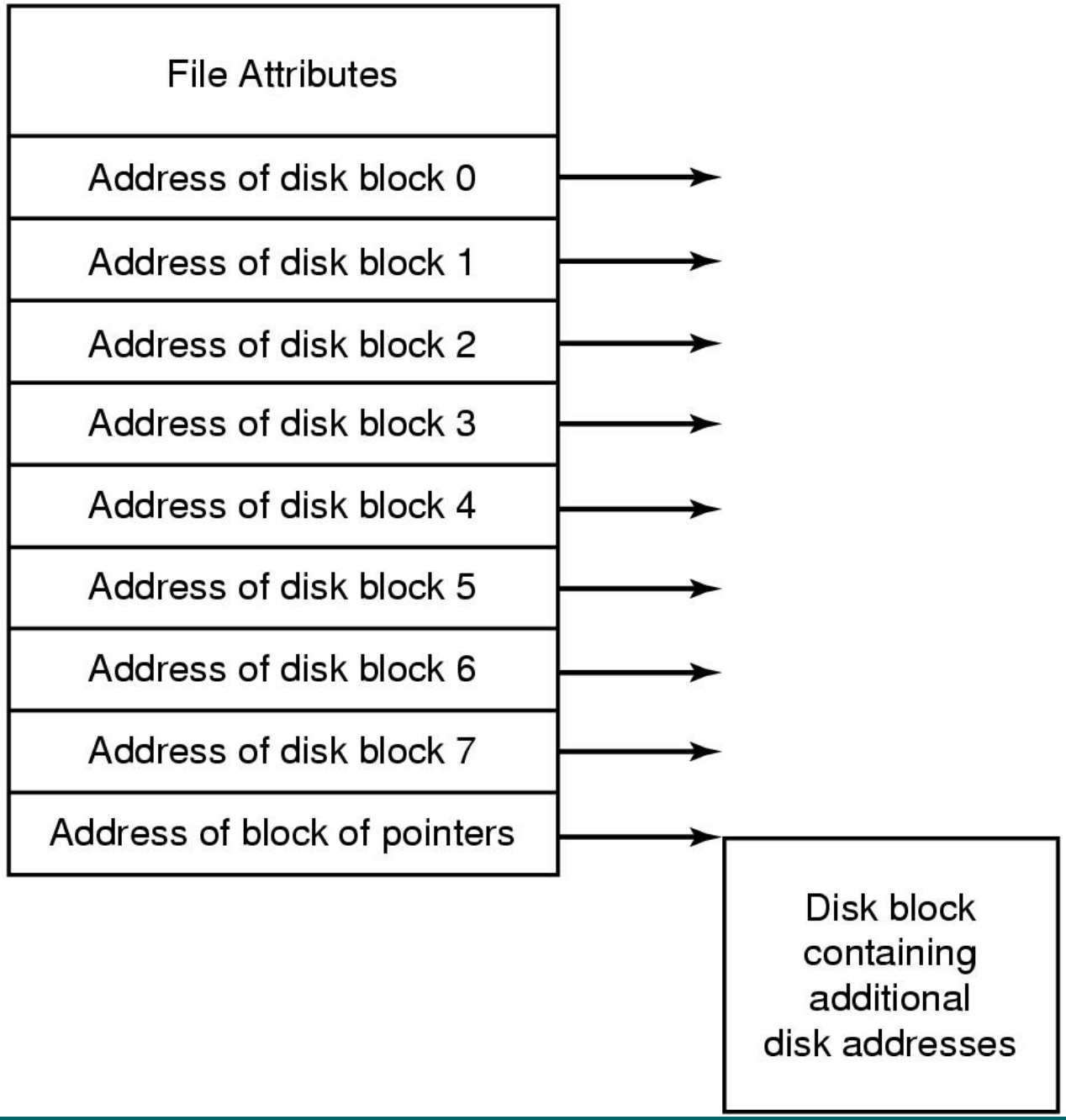


I-node structure

- mode: A. type: file, directory, pipe, symbolic link
  B. Access: read/write/execute (owner, group,)
- owner: who own this I-node (file, directory, ...)
- timestamp: creation, modification, access time
- size: the number of bytes
- block count: the number of data blocks
- direct blocks: pointers to the data
- single indirect: pointer to a data block which pointers to the data blocks (128 data blocks).
- Double indirect: (128*128=16384 data blocks)
- Triple indirect: (128*128*128 data blocks)

I-nodes

- To solve the problem of the large file table we can use i-node
- In this method for each file there is a table contains attributes and disk address of the blocks of that file. So if i-node occupies n bytes for k files open we have kn bytes of memory. Thus i-node depends on open files not disk size
- Problem is if each i-node has room for a fixed number of disk addresses what happens when a file grows beyond this limit?
- One solution is keeping multiple indexes in i-node.

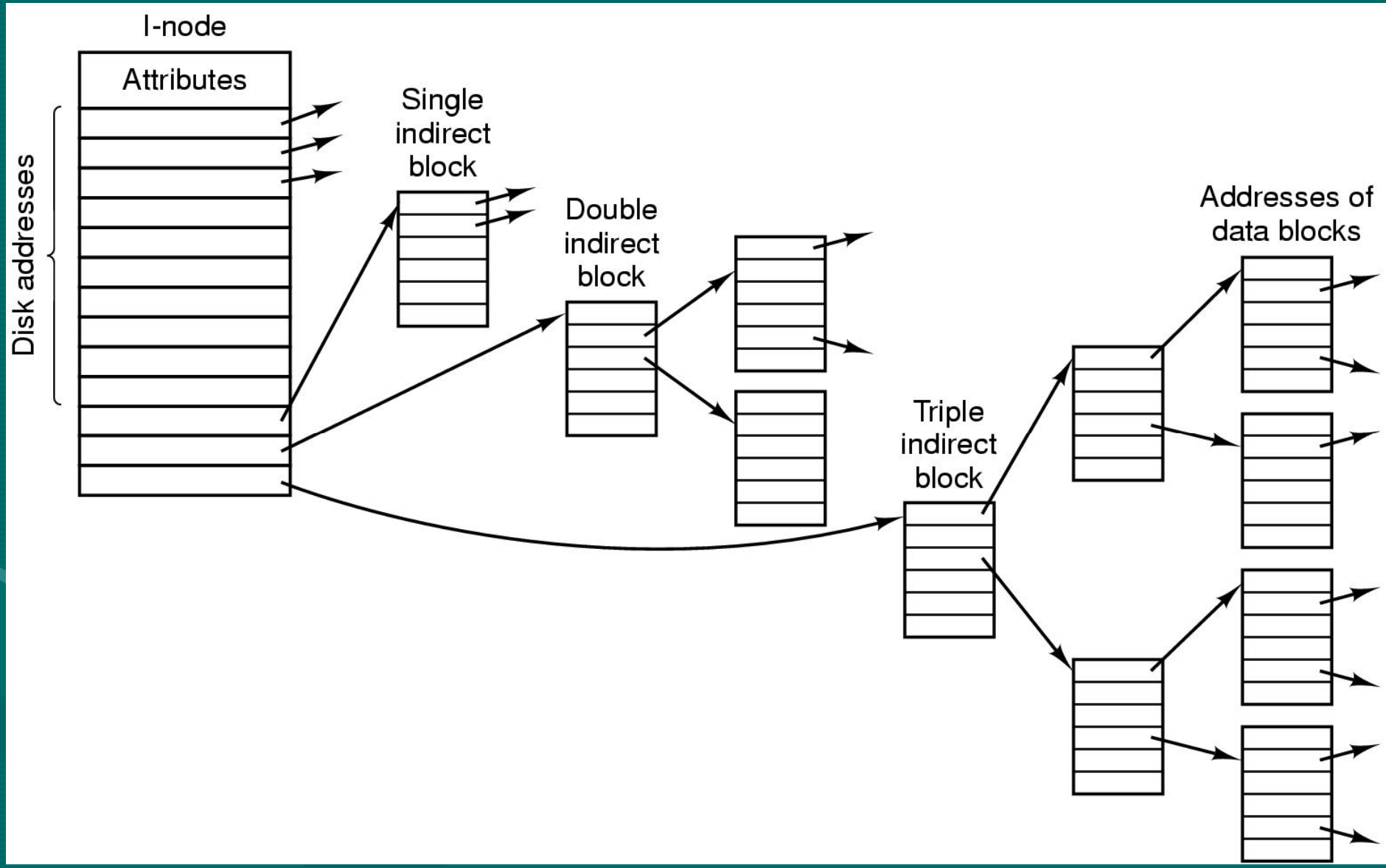

File Attributes
Address of disk block 0
Address of disk block 1
Address of disk block 2
Address of disk block 3
Address of disk block 4
Address of disk block 5
Address of disk block 6
Address of disk block 7
Address of block of pointers
Disk block containing additional disk addresses

I-node in Unix

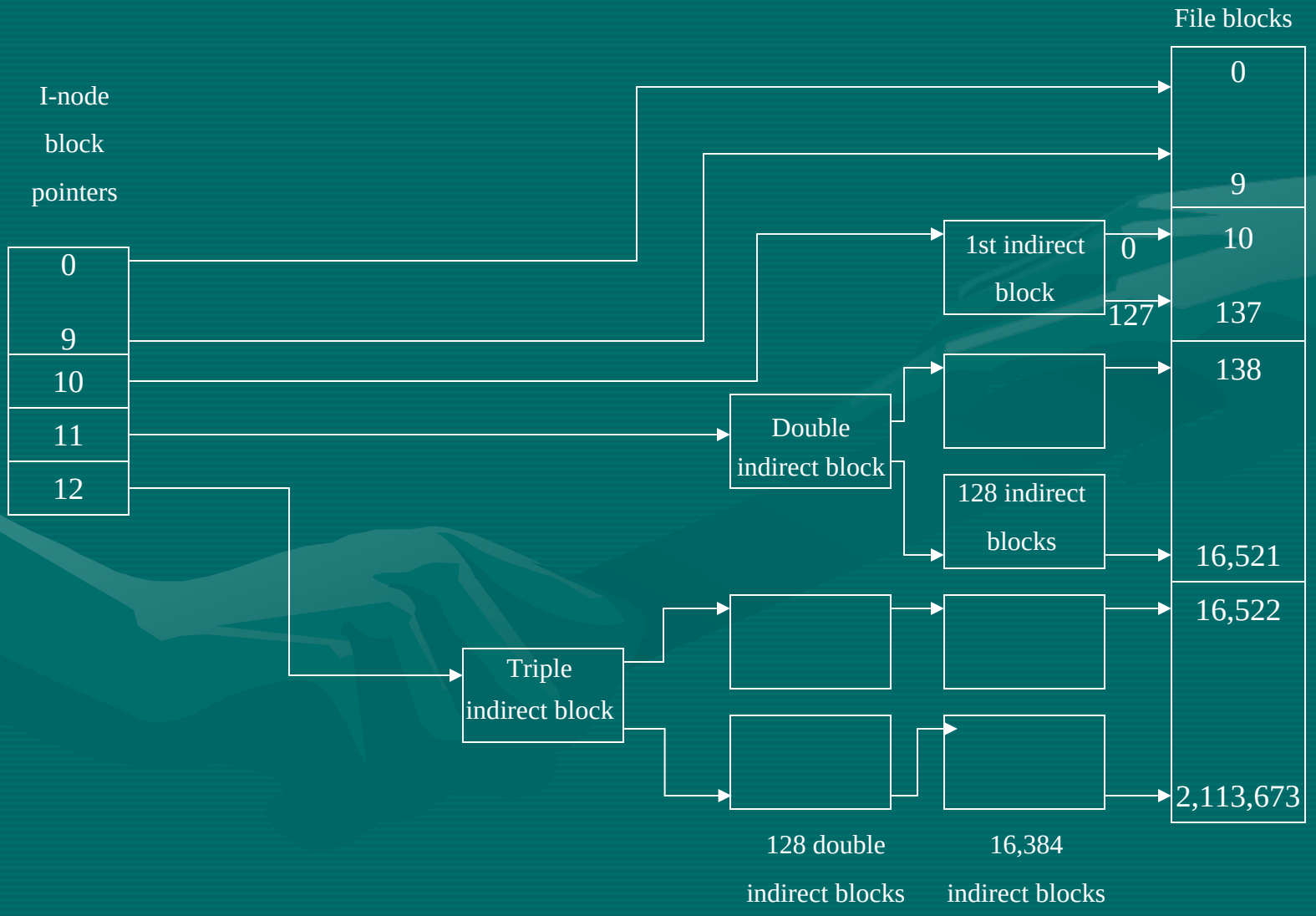
i-node in UNIX has

- Initial 10 disk addresses.
- Single indirect blocks keeps address of file more blocks for larger files.
- Double indirect block that holds address of the blocks each contains a list of single indirect block
- Triple indirect block has the address of block each is double indirect block

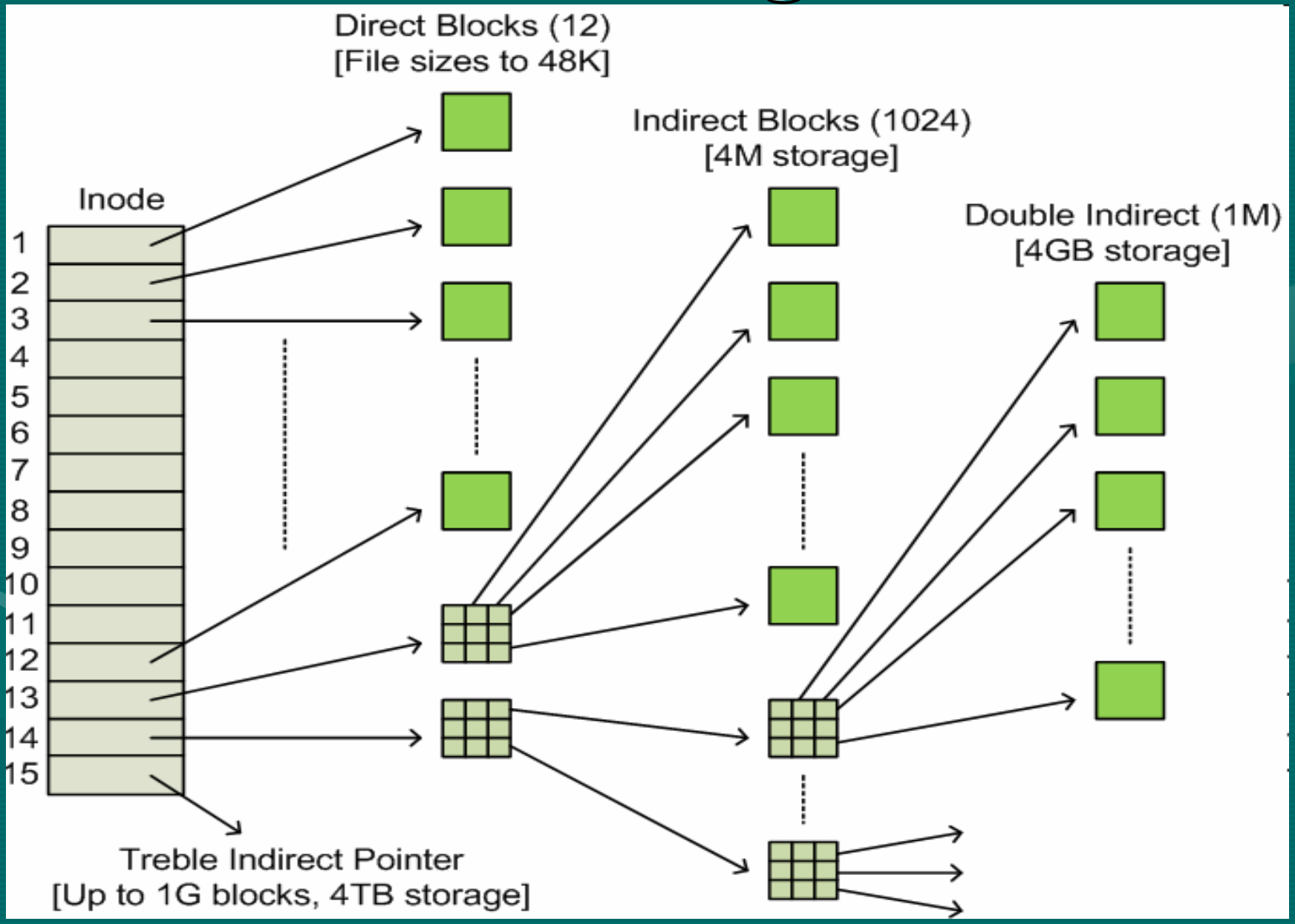
I-node in Unix



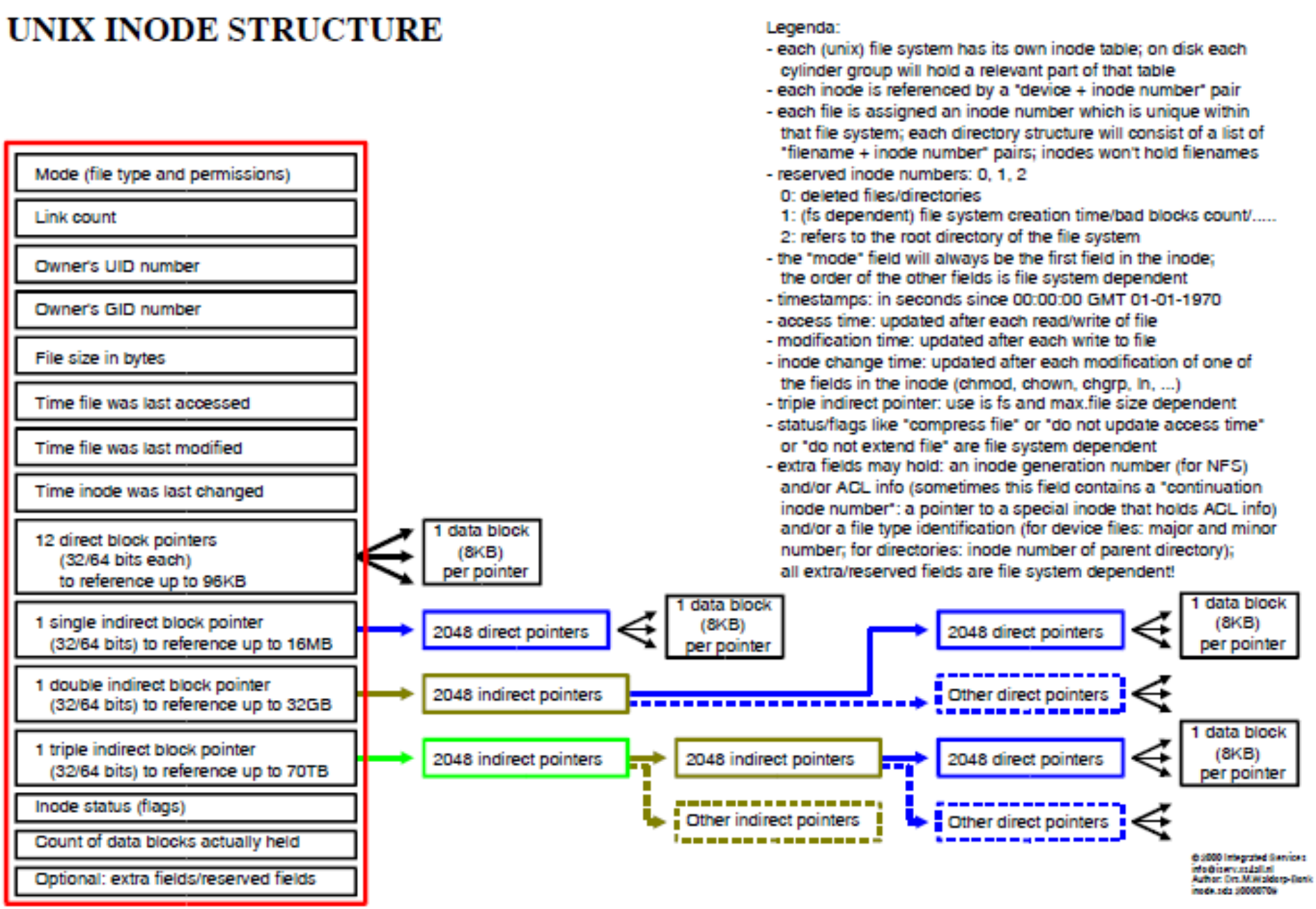
I-nodes as pointers



I-nodes Diagram



I-nodes Diagram



I-nodes Example

- A Linux file system has 4-KB blocks and 4-byte disk addresses. The *inode* of a file has 13 entries, 10 of that are direct and 3 of that are indirect and is stored in 1 block. Approximately how large is the biggest file the Linux file system can have in blocks as a power of two?
- A. 10 B. 16 C. 20 D. 30 E. 32
- Answer: D

  Explanation on next Slide

I-nodes Example

- Explanation:

  Size = size inode + size direct + size indirect + size-double indirect + size triple indirect
- inode = 1 block;
- direct = 10 blocks;
- indirect = 1 block + 2^{10} blocks (as 4K/4=1024=2^{10})
- double indirect -> indirect -> block

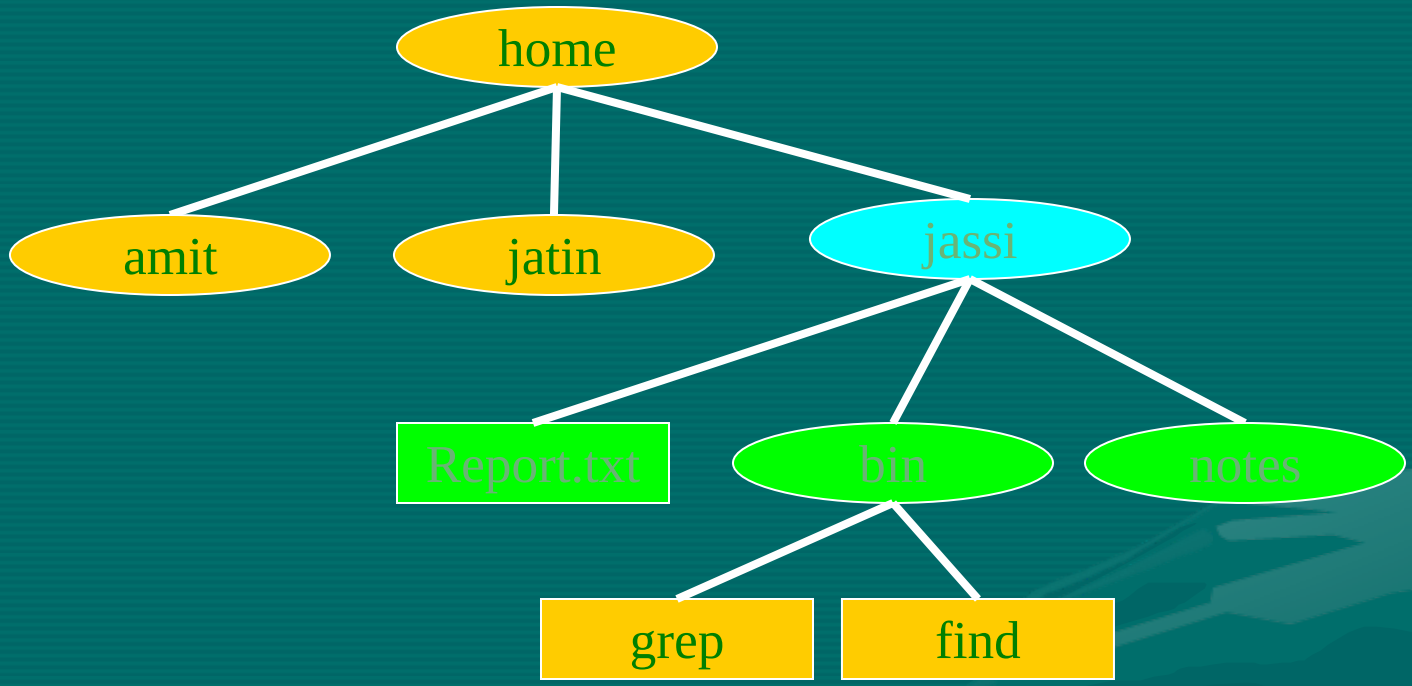
  = 1 block + 2^{10} blocks + 2^{10} x 2^{10} blocks
- Triple indirect -> double indirect -> indirect -> block

  = 1 block + 2^{10} blocks + 2^{10} x 2^{10} blocks + 2^{10} x 2^{10} x 2^{10} blocks ~ O(2^{30}) (about 1 terabyte file)

Data Block

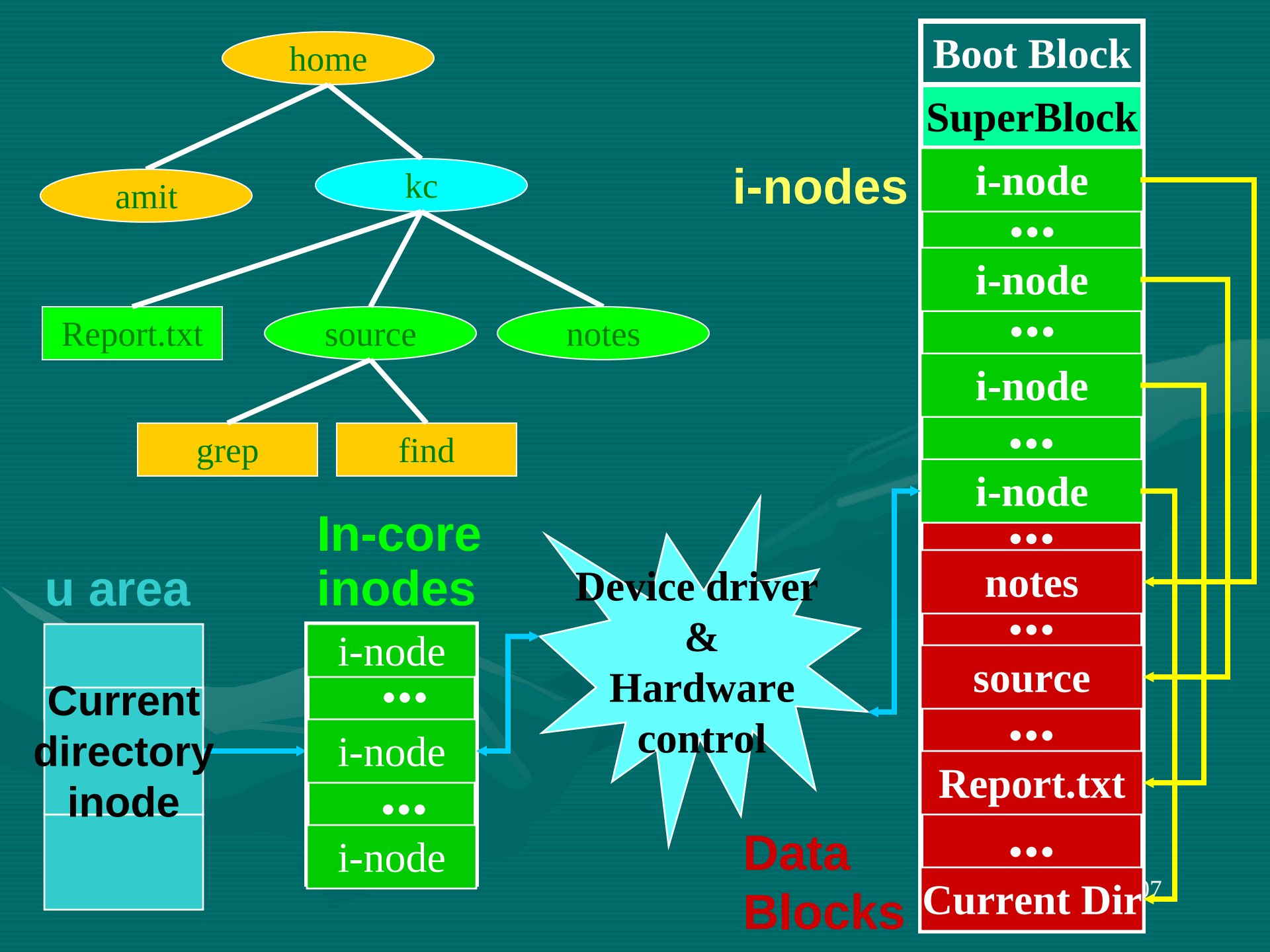
- A data block has 512 bytes.
  - A. Some FS has 1K or 2k bytes per blocks.
  - B. See blocks size effect (next page)
- A data block may contains data of files or data of a directory.
- File: a stream of bytes.
- Directory format:

i-#	Next	size	File name	pad



i-#	Next	10	Report.txt	pad	i-#	Next	3

bin	pad	i-#	Next	5	notes	pad	0	Next


home
amit
kc
Report.txt
source
notes
grep
find
i-nodes
Boot Block
SuperBlock
i-node
•••
i-node
•••
i-node
•••
i-node
•••
notes
•••
source
•••
Report.txt
•••
Current Dir
In-core inodes
u area
Current directory inode
i-node
•••
i-node
•••
i-node
Device driver & Hardware control
Data Blocks

In-core inode table

- Linux system keeps regular files and directories on block devices such as disk or tape,
- Such disk space are called physical device address space.
- The kernel deals on a logical level with file system (logical device address space) rather than with disks.
- Disk driver can transfer logical addresses into physical device addresses.
- In-core (memory resident) inode table stores the inode information in kernel space.

In-core inode table

- An in-core inode contains
  A. all the information of inode in disks.
  B. status of in-core inode
     inode is locked,
     inode data changed
     file data changed.
  C. the logic device number of the file system.
  D. inode number
  E. reference count

File table

- The kernel have a global data structure, called file table, to store information of file access.
- Each entry in file table contains:
  A. a pointer to in-core inode table
  B. the offset of next read or write in the file
  C. access rights (r/w) allowed to the opening process.
  D. reference count.